

Dart: 進階語法(2)

優雅地處理錯誤問題

- 本章節將說明如何使用 try/catch 區塊處理程式可能發生的例外狀況。
- 準備工作：
 - 將 dartex3 專案目錄，複製一份成 dartex4，做為開發專案資料夾。

```
cp -r dartex3 dartex4
```

- 複製之前，請注意目前所在的工作目錄。
- Task 目標：
 - 本章節將處理執行 command_runner 軟體套件時，所可能發生的例外狀況。
 - 利用自訂的例外狀況處理類別，妥善處理以及管理程式運行時的錯誤情況。
- Task1：建立客製化的 ArgumentError 類別
 - 首先，建立 command_runner/lib/src/exceptions.dart 檔案，內含 ArgumentError 類別
 - 檔案內容如下所示：

```
class ArgumentError extends FormatException {  
  
    /// 在發生錯誤之前，先解析 command  
    /// 假如錯誤是發生在根解析器上，此處 command 將會空白  
  
    final String? command;  
  
    /// 當錯誤發生時，將解析該參數名稱  
  
    final String? argumentName;  
  
    ArgumentError(  
        super.message, [  
            this.command,  
            this.argumentName,  
            super.source,  
            super.offset,  
        ]);  
  
    @override  
    String toString() {  
        return 'ArgumentException: $message';  
    }  
}
```

- FormatException 類別是 Dart 內建的例外處理類別。
- ArgumentError 繼承自 FormatException 類別。
- FormatException 類別包含額外的屬性，可儲存與錯誤訊息有關的 command 與 argument 變數。

- `command`：發生異常時，正在處理的指令。
 - `argumentName`：導致異常的參數名稱
- Task2：在 `CommandRunner` 中實作錯誤處理
 - 開啟 `command_runner/lib/src/command_runner_base.dart` 檔案
 - 匯入必要的函式庫 `dart:async`，以及將需要新增的 `exceptions.dart` 檔案：

```
import 'dart:async'; // 新增此行
import 'dart:collection';
import 'dart:io';

import 'arguments.dart';
import 'exceptions.dart'; // 新增此行
(以下省略....)
```

- 新增一個 `onError` 回傳函式到 `CommandRunner` 類別內：

```
(以上省略....)
class CommandRunner {

  // 新增 CommandRunner 建構式，可接受回傳函式

  CommandRunner({this.onError});

  final Map<String, Command> _commands = <String, Command>{};

  UnmodifiableSetView<Command> get commands =>
    UnmodifiableSetView<Command>(<Command>{..._commands.values});

  // 定義 onError 屬性
  FutureOr<void> Function(Object)? onError;

  (以下省略....)
}
```

- 修改 `CommandRunner` 建構式，使其可接受 `onError` 函式。
 - 這將使得程式設計師可自訂他們自己需要的例外處理程式邏輯。
- 此項變更引入了一個可為空的 `onError` 屬性。
 - `FutureOr Function(Object)?` 類型表示它是一個接受 `Object` 物件並傳回 `Future` 物件或空值的函數，其傳回值可能為 `null`。
- 更新 `run` 函式，使用 `try/catch` 語法：

```
(以上省略....)
Future<void> run(List<String> input) async {

  // 新增的 try/catch 內容

  try {
    final ArgResults results = parse(input);

    if (results.command != null) {
```

```

        Object? output = await results.command!.run(results);
        print(output.toString());
    }

    } on Exception catch (exception) {

        if (onError != null) {
            onError!(exception);
        } else {
            rethrow;
        }

    }
}
(以下省略....)

```

- 將 run 方法內部的邏輯封裝在 try/catch 程式碼區塊中。
- 當發生異常時，則該程式碼區塊會捕獲異常，並將其傳遞給 onError 回傳函數
- 如果沒有提供回傳函數，則重新拋出異常。
- 重新拋出異常會保留原始錯誤訊息和堆疊追蹤。
- 為 parse 方法新增驗證，將 command_runner_base.dart 中現有的 parse 方法替換為以下更新版本。
 - 新版包含大量檢查，一旦偵測到無效的使用者輸入，就會拋出自訂的 ArgumentError 例外。

(以上省略....)

// 該方法完整地取代

```

ArgResults parse(List<String> input) {

    ArgResults results = ArgResults();
    if (input.isEmpty) return results;

    // 假如指令無法視別，將丟出拋出異常

    if (_commands.containsKey(input.first)) {

        results.command = _commands[input.first];
        input = input.sublist(1);

    } else {

        throw ArgumentError(
            '輸入的第一個字必須為有效指令',
            null,
            input.first,
        );

    }

    // 如果提供多個指令，也要拋出異常

```

```
if (results.command != null && input.isNotEmpty && _commands.containsKey

    throw ArgumentError(
      '一次只能輸入一個指令。取得 ${input.first} 以及 ${results.command!.name}
      null,
      input.first,
    );

}

// 程式段落：處理選項，包含旗標值

Map<Option, Object?> inputOptions = {};
int i = 0;
while (i < input.length) {

  if (input[i].startsWith('-')) {

    var base = _removeDash(input[i]);

    // 假如輸入的選項無法視別，則拋出異常

    var option = results.command!.options.firstWhere(

      (option) => option.name == base || option.abbr == base, orElse: ()

        throw ArgumentError(
          '未知選項 ${input[i]}',
          results.command!.name,
          input[i],
        );

      },
    );

    if (option.type == OptionType.flag) {

      inputOptions[option] = true;
      i++;
      continue;

    }

    if (option.type == OptionType.option) {

      // 如果選項需要參數，但是並未給予，則拋出異常

      if (i + 1 >= input.length) {

        throw ArgumentError(
          '選項 ${option.name} 需要參數值',
```

```
        results.command!.name,
        option.name,
      );
    }

    if (input[i + 1].startsWith('-')) {
      throw ArgumentError(
        '選項 ${option.name} 需要參數值，但取得另一個選項 ${input[i + 1]}',
        results.command!.name,
        option.name,
      );
    }

    var arg = input[i + 1];
    inputOptions[option] = arg;
    i++;
  }

  } else {

    // 如果一個位置提供超過一個以上的參數值，則拋出異常

    if (results.commandArg != null && results.commandArg!.isNotEmpty) {
      throw ArgumentError(
        '指令最多只能有一個參數',
        results.command!.name,
        input[i],
      );
    }
    results.commandArg = input[i];
  }
  i++;
}
results.options = inputOptions;

return results;
}

String _removeDash(String input) {

  if (input.startsWith('--')) {
    return input.substring(2);
  }

  if (input.startsWith('-')) {
    return input.substring(1);
  }

  return input;
}
```

- 更新後的解析方法現在可以主動防禦錯誤輸入。具體而言，新增的 `throw` 語句可以處理幾種常見的錯誤情況：
 - 未知指令：第一個 `if/else` 程式碼區塊確保第一個參數是有效的指令。
 - 多個命令：檢查使用者是否嘗試同時執行多個命令。
 - 未知選項：如果使用者提供尚未為該命令定義的旗標或選項（例如：`--foo`），則 `firstWhere` 中的 `orElse` 參數現在會拋出例外。
 - 缺少選項值：確保選項（例如：`--output`）後面跟著一個值，而不是另一個選項或輸入結束。
 - 參數過多：強制執行一條規則，即每個指令只能有一個位置參數。
- Task3：更新 `cli.dart` 檔案，以利使用新的錯誤處理機制
 - 開啟 `cli/bin/cli.dart` 檔案
 - 更新 `main` 函式內容，使其向 `CommandRunner` 遞送 `onError` 方法函式：

```
import 'package:command_runner/command_runner.dart';

const version = '0.0.1';

void main(List<String> arguments) {

  // 新增 onError 方法函式

  var commandRunner = CommandRunner(
    onError: (Object error) {
      if (error is Error) {
        throw error;
      }
      if (error is Exception) {
        print(error);
      }
    },
  )..addCommand(HelpCommand());

  commandRunner.run(arguments);
}
(以下省略....)
```

- 這段程式碼將一個 `onError` 回傳函數傳遞給 `CommandRunner` 建構函數。
 - 如果在命令執行過程中發生錯誤，則會呼叫 `onError` 回傳函數並傳入錯誤物件。
 - 這個回傳函數會檢查錯誤是 `Error` 還是 `Exception`。
 - 如果是 `Error`，則會重新拋出該錯誤；如果是 `Exception`，則會將其輸出到螢幕上。
- Task4：更新 `command_runner` 函式庫
 - 開啟 `command_runner/lib/command_runner.dart`，新增下列內容：

```
/// 支援工作內容的函式庫

library;

export 'src/arguments.dart';
export 'src/command_runner_base.dart';
export 'src/help_command.dart';
export 'src/exceptions.dart'; // 新增此行內容
```

```
// 待辦事項：匯出所有面向此套裝軟體內容用戶端的函式庫。  
(以下省略....)
```

- 令 `ArgumentException` 可供 `command_runner` 函式庫使用。
- 這樣可以確保 `command_runner` 套件的使用者能夠取得 `ArgumentException`。
- Task5：測試新的錯誤處理
 - 開啟 Terminal 視窗並移動到 `cli` 目錄。
 - 執行下列指令：

```
dart run bin/cli.dart invalid_command
```

- 應有下列回應的內容：

```
ArgumentException: The first word of input must be a command.
```

參考文獻

- [Dart 官方教學文件](#)

Generated by [aglio](#) on 06 May 2026